CatsCo 应该成为管理调度 agent

最终的 CatsCo 不需要自己替代所有 coding agent。它应该拥有强记忆、项目理解、任务拆解、过程监督和结果验收能力，然后调用 Codex CLI、Claude Code CLI 等专业执行器完成具体工作。

定位

CatsCo 是项目管理脑、上下文脑和验收脑。

执行器

Codex CLI、Claude Code CLI、OpenCode 或其他本地/云端 coding agent。

核心能力

可观察、可取消、可验收、可沉淀记忆的任务调度。

现有代码里已有两条基础线

内部 skill-based subagent

`src/core/sub-agent-manager.ts`
`src/core/sub-agent-session.ts`
当前通过本项目自己的 `AIService`、`SkillManager`、`ConversationRunner` 跑后台 skill 任务。

external coding agent primitive

`src/runtime/external-agent/types.ts`
`src/runtime/external-agent/coding-agent-adapter.ts`
`src/runtime/external-agent/process-runner.ts`
已经有 `TaskPacket`、`ExternalAgentRegistry`、`ProcessRunner`，但还没接入主路径。

下一步不是继续扩旧 subagent，而是把两条线统一成一个 CatsCo Task Manager。

核心产品原则

- CatsCo 负责理解用户目标、拆任务、派发、追踪、验收和沉淀记忆。
- Coding CLI agent 负责具体执行，不直接绕过 CatsCo 给用户最终判断。
- 主 agent 看到的 subagent 状态是 transient context，不污染长期 transcript。
- 最终结果由主 agent 验收、转述，并决定是否进入工作记录或长期记忆。
- `AgentSession` 不直接管理子进程。
- 任务状态由独立 manager / registry 维护。
- 危险操作必须明确确认：写主仓、删除、上传、发送、改权限。
- 不要一开始做全自动多 agent 调度系统，先做可控任务能力。

目标架构：Task Manager + Agent Adapter

CatsCo Task Manager

维护 task lifecycle、parent session、working directory、进度、日志、产物、测试结果和最终摘要。

External Agent Adapters

Codex CLI adapter、Claude Code CLI adapter、OpenCode/custom adapter。统一输入 `TaskPacket`，统一输出 `TaskResult`。

Workspace Isolation

read-only review 可在当前 repo；write task 优先 git worktree 或 task dir；直接写主 repo 必须确认。

Result Review

收集 stdout/stderr、diff summary、测试结果、风险和产物，由主 agent 做验收。

LIFECYCLE

任务生命周期要清楚

created

主 agent 已创建任务，尚未选择或启动执行器。

queued

任务等待资源、确认或工作目录准备。

running

外部 coding agent 正在执行，持续记录进度和日志。

waiting

需要用户或主 agent 补充确认、权限、范围或答案。

completed / failed

任务结束，进入验收、转述、记录和必要的后续动作。

cancelled

用户或主 agent 取消任务，保留必要日志和状态。

给主 agent 和用户的界面

主 agent 工具保持少而清楚

- `spawn_task`：创建 coding / review / research task。
- `check_task`：查看进度、日志摘要和结果。
- `cancel_task`：取消任务。
- `answer_task`：任务 waiting 时补充用户确认或说明。

Dashboard Tasks 页面

- Running / Waiting / Completed / Failed。
- Agent：Codex / Claude Code / internal skill。
- Working directory / isolation mode。
- Logs、diff summary、tests、cancel、accept / archive。

强记忆应该记录什么

CatsCo 的记忆不应该保存完整 stdout，也不应该把每个执行细节都写进长期上下文。它应该保存项目管理上真正有价值的事实。

应该沉淀

- 用户目标和任务计划。
- 关键决策和约束。
- diff summary、测试结果、风险。
- 主 agent 的验收结论和后续 follow-up。

不应该默认沉淀

- 完整 stdout / stderr。
- 临时进度噪音。
- 过长的工具输出。
- 未验收的外部 agent 结论。

接下来应该怎么做

1. 先写设计文档

新增 `docs/subagent-management-plan.md`，明确产品定位、lifecycle、workspace isolation、验收和记忆边界。

2. 统一概念

把现有 skill-based subagent 和 external-agent primitive 收敛到同一个 task registry 概念。

3. 最小闭环

实现 Task Registry + External Coding Task：spawn、status、cancel、result、logs、tests summary。

4. Dashboard 接入

新增 Tasks 页面，先只观察和取消，再做 accept / archive / memory handoff。

一句话：CatsCo 的目标不是成为另一个 coding CLI，而是成为调用最强 coding CLI agent 的管理调度和项目记忆层。